

En esta unidad aprenderemos algunas funciones y librerías de Python que nos permiten hacer el tratamiento de algunos formatos textuales.

También aprenderemos a tratar todos los archivos de un directorio y de todos sus subdirectorios de manera recursiva. Dedicaremos una sección a la descarga de contenido de la web y su conversión a texto.

De esta unidad tienes los siguientes archivos disponibles:

- Este capítulo en pdf: [09-Tratamiento_de_formatos-spa.pdf](#)
- Los programas y archivos necesarios: [programas9-spa.zip](#)

9.1. Código de caracteres: detección, tratamiento y conversión

Para la detección de la codificación de caracteres en Python podemos utilizar la librería de Python chardet. Si no la tienes instalada, puedes hacerlo utilizando pip:

```
pip install chardet
```

Recuerda que en Linux es posible que tengas que ejecutar la orden con permisos de administrador y que probablemente tengas tanto Python 2 como Python 3. En estos casos tienes que escribir:

```
sudo pip3 install chardet
```

Ahora podemos escribir un programa (programa-9-1.py) para detectar la codificación de un archivo de texto de la siguiente manera:

```
import sys
import chardet
archivo_entrada=sys.argv[1]
raw_data=open(archivo_entrada,"rb").read()
codificacion=chardet.detect(raw_data)
print("Archivo:",archivo_entrada,"Codificación:",codificacion)
print(codificacion["encoding"])
```

Como parámetro de entrada le daremos el archivo del que queremos conocer la codificación (por ejemplo, el archivo noticia2-jap.txt)

```
python3 programa-9-1.py noticia2-jap.txt
```

Que nos retornará por pantalla:

```
Archivo: noticia2-jap.txt Codificación: {'language': 'Japanese', 'confidence': 0.99, 'encoding': 'ISO-2022-JP'}
ISO-2022-JP
```

Ten en cuenta que en la variable codificación el paquete chardet nos da una diccionario con tres claves: encoding, confidence y language. En el primer print escribimos directamente todo el diccionario y en cambio en el segundo print escribimos sólo la codificación. Prueba el programa con el resto de lenguas y versiones (noticia-cat.txt, noticia2-cat.txt, noticia-rus.txt, noticia2-rus.txt y noticia-jap.txt). Observarás que el programa no siempre es capaz de detectar la lengua.

Hay otras opciones válidas para detectar la codificación de caracteres de un archivo de texto. En el programa-9-2.py utilizamos la librería magic. Recuerda que puedes instalarla con pip o pip3:

```
pip install python-magic
```

Y el programa queda como sigue:

```
import sys
import magic
```

```
fitxer_entrada=sys.argv[1]
m = magic.from_file(fitxer_entrada)
print(m)
```

Prueba esta librería con los archivos de noticias adjuntos a esta unidad.

Recordemos que una vez que conocemos la codificación de un archivo lo podremos abrir correctamente utilizando la librería codecs:

```
import codecs
entrada=codecs.open("noticia-cat","r",encoding="utf-8")
```

Hay tres modos para abrir un archivo, dependiendo de la operación que queramos hacer con él:

- r: lectura
- w: escritura
- a: para añadir datos al final del archivo
- r +: tanto para leer como para escribir

Ahora, si queremos abrir y leer un archivo de texto, haremos:

```
import codecs
entrada=codecs.open("noticia-cat.txt","r",encoding="utf-8")
for linia in entrada:
    linia=linia.rstrip()
    print(linia)
```

Cuando hacemos rstrip() en la línea estamos eliminando el carácter o caracteres de salto de línea que pueda tener. Ahora podemos abrir el archivo en la codificación adecuada y guardar su contenido en otro archivo con otra codificación (programa-9-3.py):

```
import codecs
entrada=codecs.open("noticia-cat.txt","r",encoding="utf-8")
salida=codecs.open("noticia3-cat.txt","w",encoding="iso-8859-1")
for linia in entrada:
    salida.write(linia)
```

Fíjate que ahora no es necesario utilizar rstrip() ya que cuando grabamos la línea en el archivo de salida que hacer queremos que haya el salto de línea al final de la línea.

En Python también disponemos de librerías específicas para poder detectar la lengua de un documento. Por ejemplo, langdetect. En el programa-9-4.py Podemos ver un ejemplo de utilización de esta librería. (recuerda instalar langdetect con pip si todavía no la tienes instalada)

```
from langdetect import detect
```

```
lang = detect("Metges de família que han de visitar nens perquè els serveis de pediatria del CAP estan desbordats i urgències hospitalàries amb el doble d'activitat. Com cada tardor, el virus respiratori sincicial (VRS), causant de la bronquiolitis, ja fa setmanes que circula i ja ha assolit els nivells d'epidèmia, de manera que ha tensionat els serveis de pediatria dels CAP i els hospitals, com passa també cada temporada.")
print("Lengua texto 1",lang)
```

```
lang = detect("Телеканал ссылается на заявку на исследование в Научно-техническую организацию НАТО, к которой получил доступ. Недавние учения на восточной границе НАТО, присутствие в Прибалтике и война на Украине продемонстрировали успешность российской стратегии радиоэлектронной борьбы (РЭБ). И потенциал радиоэлектронных атак (например, подавления), и радиоэлектронное обеспечение (например, обнаружение целей) показывают, что существующая инфраструктура тактической связи НАТО уязвима перед средствами РЭБ и столкнется с серьезными угрозами, - говорится в документе, передает RT.")
print("Lengua texto 2",lang)
```

El programa ofrece la siguiente salida por pantalla:

```
Lengua texto 1 ca
Lengua texto 2 ru
```

Ejercicio 9.1. Crea un programa que convierta un archivo de texto en cualquier codificación en un archivo de texto en Unicode UTF-8. Puedes descargar la solución (solucion-9-1.py), pero primero intenta hacerlo, ya que no es demasiado complicado

Ejercicio 9.2. Modifica el programa anterior de modo que si el archivo ya está en Unicode UTF-8 el programa avise y no haga nada (es decir, que no cree el archivo de salida). Puede descargar la solución (solucion-9-2.py), pero primero intente hacerlo, ya que no es demasiado complicado

9.2. Tratamiento de directorios

Trataremos todos los archivos de un directorio (al descomprimir el zip que contiene los archivos de esta unidad se habrá creado un directorio que se llama *directorio*). Podemos ver este tratamiento en el programa-9-5.py:

```
import os
for file in os.listdir("./directorio"):
    print(file)
```

También se pueden tratar los directorios de forma recursiva, es decir, recorriendo toda la estructura de subdirectorios (al descomprimir el zip que contiene los archivos de esta unidad se habrá creado un directorio que se llama *directorio-recursivo*) (programa-9-6.py):

```
import os
for root, dirs, files in os.walk("./directorio-recursivo"):
    for file in files:
        print(os.path.join(root, file))
```

Podemos forzar que solo considere los archivos con una determinada extensión (por ejemplo .txt) haciendo (programa-9-6b.py):

```
import os
for root, dirs, files in os.walk("./directorio-recursivo"):
    for file in files:
        if file.endswith(".txt"):
            print(os.path.join(root, file))
```

Ejercicio: Escribe un programa que convierta todos los archivos con extensión .txt que no estén en codificación Unicode UTF-8 en Unicode UTF-8. Si algún archivo ya lo está, no es necesario que lo convierta. Haz que el archivo de salida tenga la extensión .utf8. (Los archivos necesarios son los mismos de antes).

9.3. Tratamiento de archivos tabulados y CSV

El tratamiento de archivos de texto con campos separados por tabuladores se puede hacer fácilmente con la función estándar `split()`. Tomemos como ejemplo el archivo `cdlconsulteca.txt` que contiene entradas terminológicas con la siguiente información:

```
abdicar v tr abdicar abdicare Dret civil Renunciar al domini d'alguna cosa, una propietat, un dret o una opinió.
Consulteca-13
```

y cada uno de los campos está separado por tabulador. Hacemos un programa que lea este archivo y guarde otro que contenga solo: denominación en inglés, denominación en español, y el área de especialidad (fíjate que son el 4º campo, el 3º campo y el 5º campo (recuerda que los índices de una lista empiezan por 0, no por 1)). Podemos encontrar la solución en el programa programa-9-7.py:

```
import codecs
entrada=codecs.open("cdlconsulteca.txt","r",encoding="utf-8")
salida=codecs.open("cdlconsulteca-mod.txt","w",encoding="utf-8")

for linea in entrada:
    linea=linea.rstrip()
    campos=linea.split("\t")
    cadena=campos[3]+"\\t"+campos[2]+"\\t"+campos[4]
    print(cadena)
    salida.write(cadena+"\n")
```

Ejercicio: Este programa tiene el problema de que si una entrada no tiene denominación inglesa igualmente escribe la salida. Modifica el programa para que solo escriba la salida si tenemos tanto la denominación inglesa como la catalana.

Los archivos CSV (*Comma Separated Values*) se pueden tratar de una manera genérica utilizando el paquete csv. Estos archivos se caracterizan por:

- Un separador: , u otro
- Un delimitador de texto: " o ' o cualquier otro carácter

El archivo de texto tabulado es un caso especial donde el separador es un tabulador y no hay delimitador. Si ahora tenemos el archivo de glosario en csv utilizando comas y sin delimitador de texto (cdlconsulteca.csv). Es decir, tiene la siguiente forma:

```
abdicar,v tr,abdicar,abdicate,Dret civil,"Renunciar al domini d'alguna cosa, una propietat, un dret o una opinió.",Consulteca-13
```

Podemos hacer el tratamiento de este archivo con el paquete csv haciendo (programa-9-8.py):

```
import csv
import codecs
entrada=codecs.open("cdlconsulteca.csv","r",encoding="utf-8")
lector=csv.reader(entrada, delimiter=',', quotechar='')
for campos in lector:
    print(campos)
```

El paquete csv también nos proporciona funcionalidades para escribir archivos csv.

```
escriptor = csv.writer(csvfile, delimiter=';', quotechar='"', quoting=csv.QUOTE_MINIMAL)
```

- sv.QUOTE_ALL: Instructs writer objects to quote all fields.
- csv.QUOTE_MINIMAL : Instructs writer objects to only quote those fields which contain special characters such as delimiter, quotechar or any of the characters in lineterminator.
- csv.QUOTE_NONNUMERIC: Instructs writer objects to quote all non-numeric fields.
- csv.QUOTE_NONE: Instructs writer objects to never quote fields. When the current delimiter occurs in output data it is preceded by the current escapechar character. If escapechar is not set, the writer will raise Error if any characters that require escaping are encountered.

Podemos hacer por ejemplo un programa que convierta el archivo csv del glosario en otro archivo donde el separador sea punto y coma (;) y el delimitador de texto sean las comillas ("). Lo podemos hacer con el programa-9-9 .py:

```
import csv
```

```
import codecs
entrada=codecs.open("cdlconsulteca.csv","r",encoding="utf-8")
lector=csv.reader(entrada, delimiter=',', quotechar="")
salida=codecs.open("cdlconsulteca2.csv","w",encoding="utf-8")
escriptor = csv.writer(salida, delimiter=';', quotechar="", quoting=csv.QUOTE_ALL)
for linea in lector:
    escriptor.writerow(linea)
```

Si ejecutamos este programa, en el archivo de salida (cdlconsulteca2.csv) obtendríamos la información de la siguiente manera:

```
"abdicar";"v tr";"abdicar";"abdicate";"Dret civil";"Renunciar al domini d'alguna cosa, una propietat, un dret o una opinió."; "Consulteca-13"
```

9.4. Obtención de información de la web

Con Python podemos obtener fácilmente información de la web. Por ejemplo, para obtener una web en formato html (programa-9-10.py) (NOTA: pon una dirección web completa en la tercera línea)

```
import urllib.request
import codecs
f = urllib.request.urlopen("https://www.ara.cat/esports/barca/resultats-observatori-blaugrana-febrer-2018_0_1954004729.html")
web=f.read().decode('utf-8')

sortida=codecs.open("noticia.html","w",encoding="utf-8")
sortida.write(web)
```

En el programa-9-11.py obtenemos esta información en html y la convertimos en texto.

```
import urllib.request
import codecs
import html2text
f = urllib.request.urlopen("https://www.ara.cat/esports/barca/resultats-observatori-blaugrana-febrer-2018_0_1954004729.html")
web=f.read().decode('utf-8')
text=html2text.html2text(web)
sortida=codecs.open("noticia.txt","w",encoding="utf-8")
sortida.write(text)
```